SE327 Term Project Report: Excalibur Technology Re-engineering

We rebuilt Excalibur, our SE262 game-key web store, with a new technology in every component: **Tailwind CSS + DaisyUI** (UI), **React with Vite** (frontend), **Fastify** (backend) and **PostgreSQL** (database). The new version keeps every SE262 feature, adds 125 automated tests, and passes our SonarQube quality gate, which the original code fails.

1. Introduction

This project re-engineers our SE262 web store with four new technologies and measures how the choices changed the code, the process and the product.

Team

Member	Student ID	Responsibility
Piyachat Kamklat	672115030	UI framework, frontend, frontend tests, SonarQube for UI/frontend, report and slides
Aung Khaing Phyo	672115501	Backend, database, backend tests, CI, SonarQube for the whole project

The original application. Excalibur is a store for digital game keys. Customers browse 114 games across 10 platforms (Steam, Xbox Live, Nintendo and others), add keys to a cart, pay by PromptPay QR or TrueMoney, receive their keys, and write reviews. Admins manage products, categories, game keys, users, orders, payments and reviews from a back office with a sales dashboard.

Project goals

1. Rebuild the store so it works at least as well as the SE262 version.
2. Replace all four components with technologies we did not use in SE262.
3. Measure the result with tests, coverage and SonarQube, and use a quality gate during development.
4. Compare the old and new versions and recommend a stack for similar projects.

Component	SE262 (original)	SE327 (re-engineered)
UI framework	Bootstrap 5 + custom CSS	Tailwind CSS 4 + DaisyUI 5
Frontend	EJS server-rendered templates + vanilla JavaScript	React 19 single-page app built with Vite 7
Backend	Node.js with Express 5	Fastify 5 JSON REST API
Database	MySQL 8	PostgreSQL 16

The new code is at github.com/A2-HrMSTrailblazer/excalibur-v2.

the original is at github.com/A2-HrMSTrailblazer/termproject-2025-excalibur.

2. Technology survey: selection criteria

For each component we surveyed three alternatives that differ from the SE262 technology, and scored each from 1 (poor) to 5 (excellent) on the same five weighted criteria.

Criterion	Weight	What it measures
Speed of rewrite	30%	How fast two students can rebuild every SE262 feature with it
Fit with project goals	25%	Can it reproduce the SE262 look and behaviour, including admin tables, checkout and key assignment?
Learning curve	20%	New knowledge needed, given our SE262 experience with HTML, Bootstrap, JavaScript, Express and SQL
Testability and SonarQube support	15%	Can we write automated tests and can SonarQube analyse it? (graded in the Process part)
Ecosystem and community	10%	Documentation, examples and long-term support

Each option is also judged on four questions the assignment asks: its **strengths**, its **weaknesses**, its **suitability for our project goals** and its **suitability for our personal preferences**.

3. Survey: UI framework

Tailwind CSS + DaisyUI won (4.65 / 5) because it rebuilds the SE262 look fastest and does not lock us into one frontend framework. SE262 used Bootstrap 5, so all three options are new to the project.

Tailwind CSS + DaisyUI

- **Strengths:** utility classes work with any frontend; only the classes used end up in the CSS bundle; DaisyUI component classes (`btn`, `card`, `modal`, `table`, `drawer`) map closely to Bootstrap's; themes define the SE262 colours (`#00001A`, `#222584`, `#FFCC00`) once.
- **Weaknesses:** long class lists make markup harder to read; many utility names to learn; DaisyUI is CSS only, so we write modal and dropdown behaviour ourselves.
- **Fit with project goals:** very high. A dark storefront theme and a light admin theme are both built in, and no runtime library is added.
- **Fit with our preferences:** high. We already think in Bootstrap-style classes and want full control of the look.

Material UI (MUI)

- **Strengths:** the richest set of complete React components (DataGrid, Dialog, Date Picker); strong accessibility; mature documentation.
- **Weaknesses:** React only; the Material look is hard to restyle into the SE262 gaming style; larger bundle and a styling engine that runs in the browser.
- **Fit with project goals:** medium. Admin tables would be quick, but matching the storefront design needs many theme overrides.
- **Fit with our preferences:** medium. We would spend time fighting the default design.

Chakra UI

- **Strengths:** readable style props (`<Box p={4}>`); good accessibility and dark-mode defaults; easy theming.

- **Weaknesses:** React only; smaller ecosystem; a recent major version changed the API, so many tutorials are outdated.
- **Fit with project goals:** good. Theming is easier than MUI, but data tables need extra libraries.
- **Fit with our preferences:** medium to high, but outdated examples are risky under a deadline.

Criterion (weight)	Tailwind + DaisyUI	Material UI	Chakra UI
Speed of rewrite (30%)	5	3	4
Fit with project goals (25%)	5	3	4
Learning curve (20%)	4	3	4
Testability and SonarQube (15%)	4	4	4
Ecosystem (10%)	5	5	3
Weighted total	**4.65**	**3.35**	**3.90**

4. Survey: frontend framework

React won (4.80 / 5), just ahead of Vue 3 (4.45), on ecosystem, testing tools and full SonarQube support. SE262 used server-rendered EJS templates, so all three options are new.

React (with Vite)

- **Strengths:** largest ecosystem of libraries and answers; React Router, react-chartjs-2 and React Testing Library cover every SE262 need; JSX is fully analysed by SonarQube; Vite starts instantly.
- **Weaknesses:** only a view library, so routing and state need extra choices; hook rules (`useEffect`) are easy to get wrong; JSX mixes markup and logic.
- **Fit with project goals:** very high. Each SE262 page maps to a component that can be tested the way a user clicks it.
- **Fit with our preferences:** high. It is the framework we most want to build experience in for internships.

Vue 3

- **Strengths:** HTML-like templates closest to EJS, so the gentlest learning curve; official router and state library; `.vue` files are supported by SonarQube.
- **Weaknesses:** smaller ecosystem than React; two API styles split the tutorials; fewer local job openings.
- **Fit with project goals:** very high, equal to React.
- **Fit with our preferences:** medium to high. Easy, but less useful for our plans.

Svelte (SvelteKit)

- **Strengths:** least code to write; a compiler, so small and fast output with no framework runtime.
- **Weaknesses:** smallest ecosystem and fewest testing examples; we found no official SonarQube support for `.svelte` files at the time of the survey.
- **Fit with project goals:** good for the pages, weak for the SonarQube and coverage requirements.
- **Fit with our preferences:** medium. Interesting, but risky when SonarQube results are graded.

Criterion (weight)	React (Vite)	Vue 3	Svelte
Speed of rewrite (30%)	5	4	3
Fit with project goals (25%)	5	5	4
Learning curve (20%)	4	5	4
Testability and SonarQube (15%)	5	4	3
Ecosystem (10%)	5	4	3
Weighted total	**4.80**	**4.45**	**3.45**

5. Survey: backend framework

Fastify won (4.70 / 5) because its routing is close to Express, so SE262 business logic could be ported quickly, while adding schema validation and built-in test injection. SE262 used Express 5, so all three options are new.

Fastify

- **Strengths:** route handlers look like Express, so porting is fast; JSON Schema validation on every request body; official plugins for sessions, file upload and static files; `app.inject()` runs HTTP tests without opening a port; one of the fastest Node.js frameworks.
- **Weaknesses:** plugin and encapsulation rules take time to understand; smaller ecosystem than Express; errors from schema validation need a custom handler to be user-friendly.
- **Fit with project goals:** very high. Every SE262 route became a JSON endpoint, and checkout could be written as one database transaction.
- **Fit with our preferences:** high. Familiar JavaScript style, with better structure and tests than our Express code.

NestJS

- **Strengths:** a full framework with modules, controllers, dependency injection and TypeScript; excellent testing support; popular in companies.
- **Weaknesses:** a lot of boilerplate (modules, decorators, providers) for a small store; TypeScript and dependency injection are a steep learning curve for us.
- **Fit with project goals:** high for a large team, but too heavy to finish in our time.
- **Fit with our preferences:** medium. Good for the CV, but we would spend most of the time learning the framework.

Hono

- **Strengths:** very small and fast; runs on Node.js, Bun and edge platforms; clean, modern API; tests with `app.request()`.
- **Weaknesses:** younger ecosystem; fewer ready-made plugins for sessions and file uploads on Node.js; fewer examples of full server-rendered or session-based apps.
- **Fit with project goals:** good, but sessions and image upload would need more custom code.
- **Fit with our preferences:** medium to high. Pleasant to write, but less proven for our use case.

Criterion (weight)	Fastify	NestJS	Hono
Speed of rewrite (30%)	5	2	4
Fit with project goals (25%)	5	5	4
Learning curve (20%)	4	2	4
Testability and SonarQube (15%)	5	5	4
Ecosystem (10%)	4	5	3
Weighted total	**4.70**	**3.50**	**3.90**

6. Survey: database

PostgreSQL won (4.65 / 5) because our data is relational (games, keys, orders, payments) and checkout needs safe transactions; it keeps the SQL we know while being stricter than MySQL. SE262 used MySQL 8, so all three options are new.

PostgreSQL

- **Strengths:** full SQL with strict typing; named ENUM types; views and `LATERAL` joins; `FOR UPDATE SKIP LOCKED` so two buyers never receive the same key; mature and free.
- **Weaknesses:** some MySQL habits break (reserved words `user` and `order`, case-sensitive `LIKE`, `?` placeholders become `$1`); the Node.js driver returns `NUMERIC` as a string.
- **Fit with project goals:** very high. The SE262 schema and seed data (114 games, 570 keys) could be converted rather than redesigned.
- **Fit with our preferences:** high. We keep using SQL and learn the database most used in industry tutorials.

MongoDB

- **Strengths:** flexible JSON documents; easy to start; very large community and cloud hosting.
- **Weaknesses:** our data is highly relational (an order item points to a key, which points to a game and a category), so it would need a full redesign; multi-document transactions are more complex than in SQL.
- **Fit with project goals:** low to medium. Feature parity would take much longer.
- **Fit with our preferences:** medium. Interesting to learn, but a poor match for this data.

SQLite

- **Strengths:** no server to install, a single file; very fast for small apps; ideal for tests.
- **Weaknesses:** one writer at a time, so busy checkouts would queue; fewer data types and no ENUM; less like a production setup.
- **Fit with project goals:** medium. Works for a demo, but weaker for concurrent checkout and admin use.
- **Fit with our preferences:** medium. Simple, but teaches us less than a client-server database.

Criterion (weight)	PostgreSQL	MongoDB	SQLite
Speed of rewrite (30%)	5	2	4
Fit with project goals (25%)	5	3	3
Learning curve (20%)	4	3	5

Criterion (weight)	PostgreSQL	MongoDB	SQLite
Testability and SonarQube (15%)	4	4	5
Ecosystem (10%)	5	5	4
Weighted total	**4.65**	**3.05**	**4.10**

7. Selected stack and justification

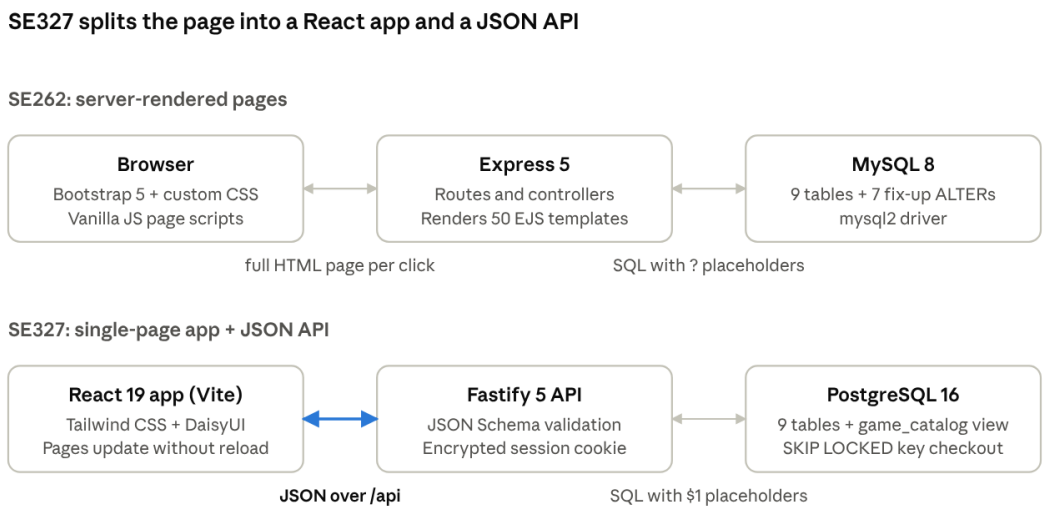
In every component the winner scored highest mainly on **speed of rewrite** and **fit with project goals**, the two criteria worth 55% together. None of the four choices is the SE262 technology.

Component	Choice	Score	Runner-up	Score	Deciding reason
UI framework	Tailwind CSS + DaisyUI	4.65	Chakra UI	3.90	Fastest way to recreate the SE262 theme; works with any frontend
Frontend	React (Vite)	4.80	Vue 3	4.45	Largest ecosystem, best testing tools, full SonarQube support
Backend	Fastify	4.70	Hono	3.90	Express-like routes for fast porting, plus schema validation and `inject()` tests
Database	PostgreSQL	4.65	SQLite	4.10	Relational data and safe concurrent checkout with `SKIP LOCKED`

The four choices also fit together: Tailwind does not depend on React, Vite proxies `/api` to Fastify during development, and Fastify serves the built React app in production, so the whole store runs as one Docker container next to PostgreSQL.

8. Architecture before and after

The structural change is the new JSON boundary: SE262 sent a full HTML page for every click, while SE327 sends the React app once and then exchanges only JSON with the Fastify API.



In production, Fastify also serves the built React files, so the store runs as one container beside PostgreSQL.

9. Reimplementation

Every SE262 feature exists in the new version, and none of the four components reuses SE262 technology; the code is about 40% the size of the original.

Feature parity

SE262 feature	SE327 page	API endpoint
Home (Nintendo, Top games, Recommended)	/	`GET /api/home`
Store with search, platform and price filters, 15 per page	`/store`	`GET /api/games`
Product detail with price-history chart and reviews	`/product/:id` (chart zoom 1m / 3m / All)	`GET /api/games/:id`
Cart with stock checks	`/cart`	`GET/POST /api/cart`, `PATCH/DELETE /api/cart/:gameId`
Checkout (PromptPay QR / TrueMoney, 15-minute timer)	`/checkout`	`POST /api/checkout`
Orders with delivered keys	`/orders`	`GET /api/orders`
Write a review (4 detailed ratings → overall)	`/review/:id`	`POST /api/games/:id/reviews`
Profile (change username)	`/profile`	`GET/PATCH /api/profile`
Login, register, logout	`/login`, `/register`	`/api/auth/*`
About, contact	`/about`, `/contact`	—
Admin dashboard, products, categories, game keys, users, orders, payments, reviews	`/admin/*`	`/api/admin/*`

What changed in each component

- **UI (Tailwind CSS + DaisyUI):** 3,108 lines of custom CSS in 21 files became 57 lines of theme tokens. Two DaisyUI themes reproduce the dark navy and yellow storefront and the light back office.
- **Frontend (React + Vite):** 50 EJS templates became about 40 React components and pages. The cart, checkout modal and admin tables update without reloading the page. Admin pages load only when an admin opens them.
- **Backend (Fastify):** Express controllers that rendered HTML became JSON endpoints with schema validation. Sessions are an encrypted cookie. Checkout is a single transaction that reserves real keys.
- **Database (PostgreSQL):** the MySQL `init.sql` was converted, keeping all seed data. Tables were renamed (`user` → `users`, `order` → `orders`), ENUMs became named types, and a `game_catalog` view replaced a price-and-stock sub-query that was copied into six controllers.

Bugs from SE262 fixed during the rewrite: plain-text passwords (now bcrypt), the client choosing its own price at checkout, adding and deleting game keys always failing, deleting a product always

failing, registration crashing on duplicate usernames, and dashboard revenue counting refunded orders.

10. Challenges the survey did not predict

The biggest surprises came from the frontend/backend split and from PostgreSQL being stricter than MySQL.

Area	What happened	How we solved it
Frontend	Moving from EJS to React meant designing a JSON API that SE262 never had, and handling loading states on every page	Agreed the API contract first, then built frontend and backend in parallel
Frontend	The Checkout page sent users back to the cart before the cart had finished loading	Tracked whether the cart was loaded for the current user before deciding it was empty
Frontend tests	A review test that typed 150 characters passed on one laptop but timed out on a slower Windows laptop (a flaky test)	Pasted the text instead of typing it key by key, and raised the test timeout to 20 seconds
Database	pg returns `NUMERIC` values as strings, so `price * quantity` could silently break	Added a type parser that converts them to numbers
Database	MySQL compared `'Available'` and `'available'` as equal; PostgreSQL ENUMs did not	Used exact lower-case values everywhere
Database	Seeding rows with explicit IDs left the identity counters behind, so new inserts collided	Called `setval()` after seeding
Backend	Fastify's encrypted session cookie contains characters that broke our test helper	URL-encoded the cookie value in the tests
Tooling	SonarQube cannot read `.ejs`, so part of the old frontend was invisible to analysis	Noted as a limitation of the old stack; JSX is fully analysed
Tooling	A `package-lock.json` generated on Linux made `npm install` skip the Windows build of `lightningcss` (used by Tailwind), so tests would not start on a Windows laptop	Reinstalled cleanly on Windows and kept the shared lockfile out of the pull request

11. Process: Git workflow

We use a simple Git Flow: `master` for releases, `develop` for integration, and one short-lived branch per task, merged through a pull request. The rules are written in `CONTRIBUTING.md` in the repository.

1. **Branches:** `feature/*` for new work, `fix/*` for bugs and SonarQube findings, `test/*` for tests, `chore/*` for tooling.
2. **Pull requests:** every change goes into `develop` through a PR, using the PR template (`.github/pull_request_template.md`), which asks for lint, coverage and quality-gate checks.
3. **Review:** the other member checks the PR description, test results and SonarQube result, then merges it (PR #1 was merged by Aung Khaing Phyo).
4. **Branch protection:** `CONTRIBUTING.md` describes protection rules for `master` and `develop` (a PR, one approval and passing CI checks); turning them on in GitHub settings is our next step.
5. **Commit messages:** Conventional Commits, for example `refactor: use optional chaining for safer property access`.
6. **CI:** GitHub Actions runs lint, tests, coverage thresholds and the build on every push and PR (`.github/workflows/ci.yml`).

Commits driven by SonarQube findings

Commit	Author	Change
`ce5f85f`	Aung Khaing Phyo	`refactor: use optional chaining for safer property access in multiple components` (6 files)
`39cbf70`	Aung Khaing Phyo	`refactor: use void operator for navigate calls in Header component`
PR #1, commit 4514dd1	Piyachat Kamklat	`fix(ui): resolve SonarQube security and reliability issues` (ToastContext, Checkout, Header)

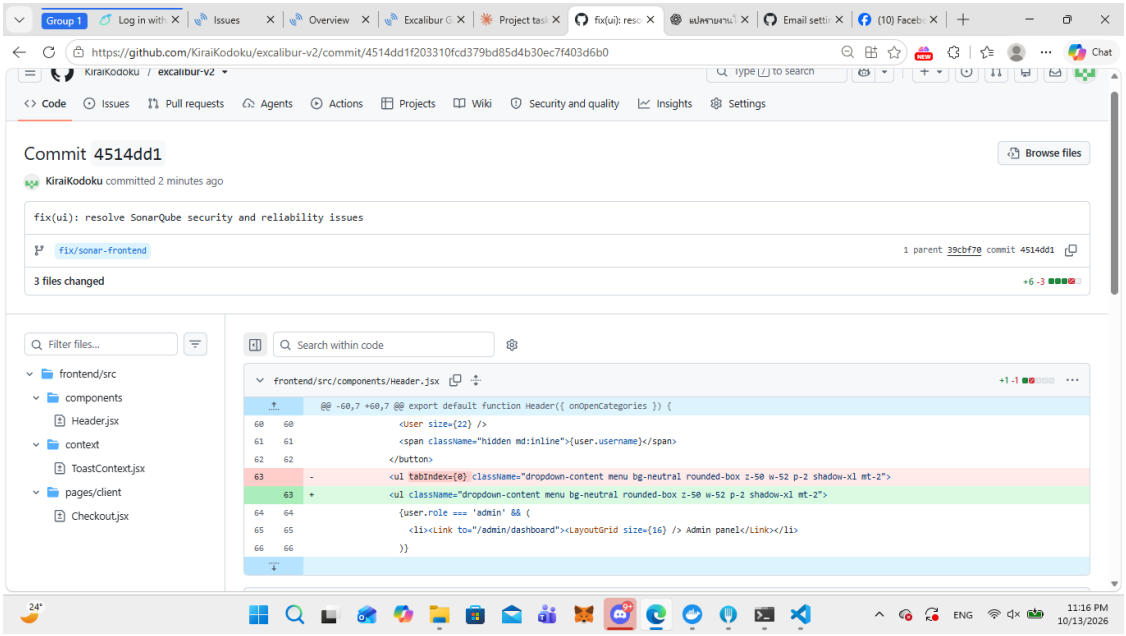


Figure 11.1: commit 4514dd1 "fix(ui): resolve SonarQube security and reliability issues" on branch `fix/sonar-frontend` in Piyachat's fork: 3 files changed (+6 −3), sent to the team repository as a pull request.

Pull requests

PR	From → into	Author	Merged by	Content
#2 Develop (release)	`develop` → `master`	Aung Khaing Phyo	A2-HrMSTrailblazer	2 commits, 3 files; 3 CI checks; SonarQube and coverage screenshots in the description
#1 fix(ui): resolve SonarQube security and reliability issues	`KiraiKodoku:fix/sonar-frontend` (fork) → `develop`	Piyachat Kamklat	A2-HrMSTrailblazer	1 commit, 3 files (+6 −3); test and SonarQube results in the description

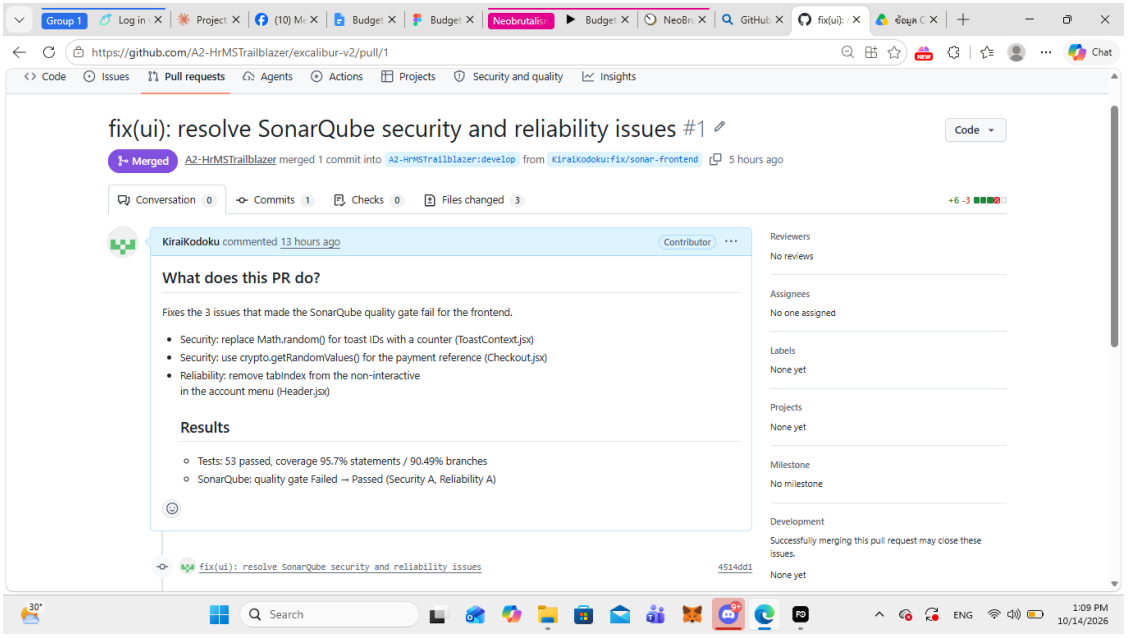


Figure 11.2: PR #1, opened from Piyachat's fork (`KiraiKodoku:fix/sonar-frontend`) and merged into `develop`, with the fix list and results in the description.

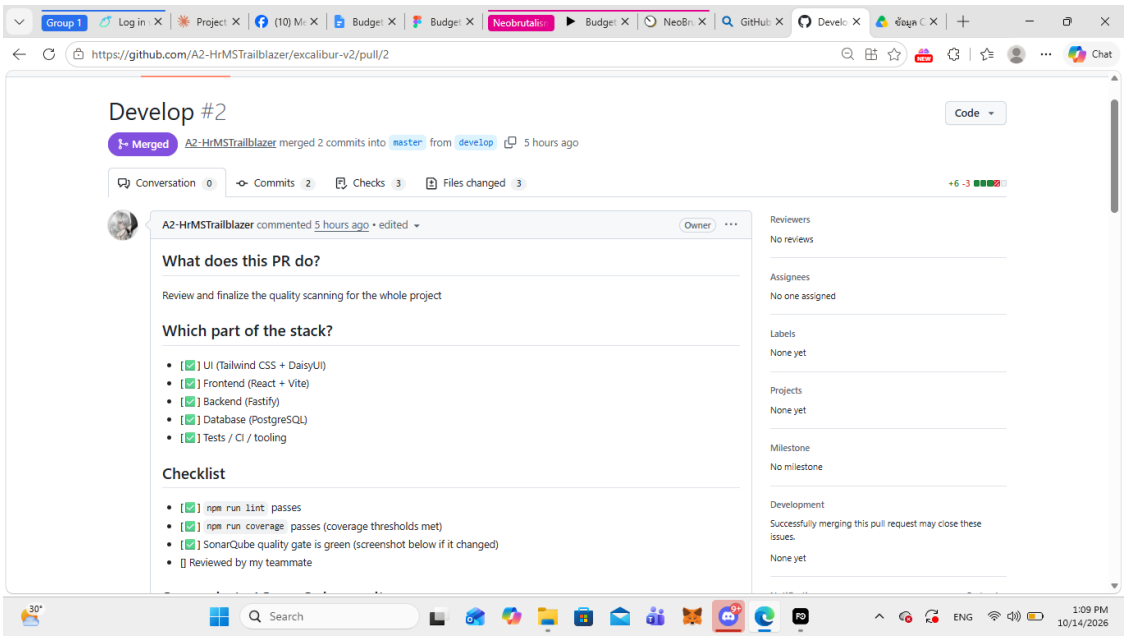


Figure 11.3: PR #2, the release from `develop` into `master`, using the PR template checklist (lint, coverage, SonarQube quality gate); 3 CI checks ran.

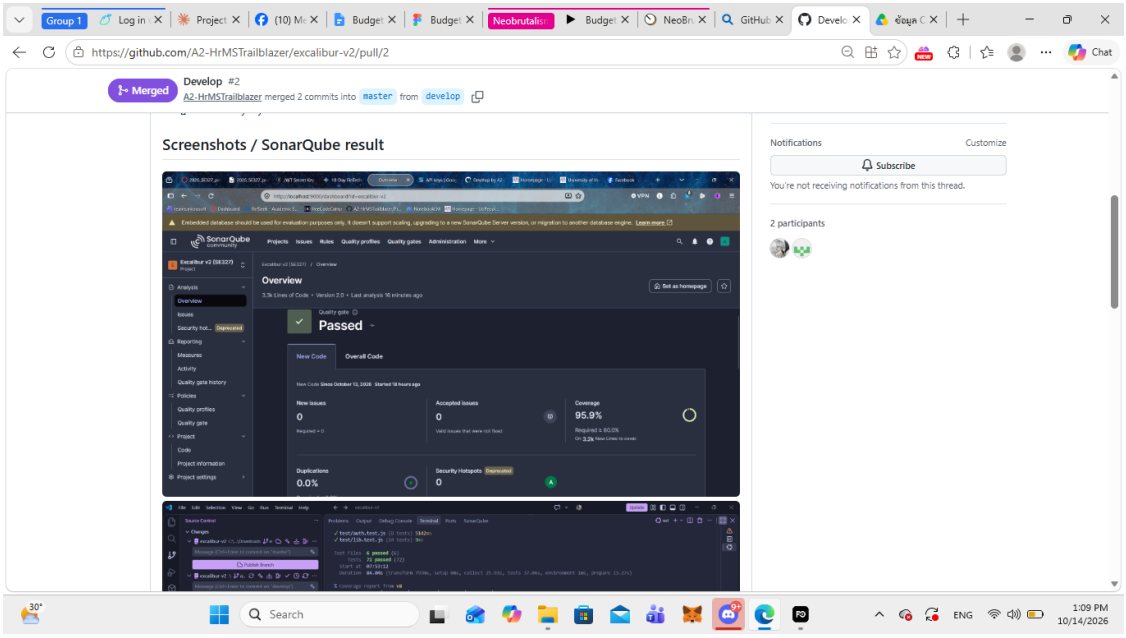


Figure 11.4: PR #2's description attaches the SonarQube result (quality gate Passed, 95.9% coverage) and the test coverage output as evidence.

12. Process: tests and statement / branch coverage

The new version has 125 automated tests with over 90% statement and branch coverage in both halves; SE262 had no tests at all.

Part	Tool	Tests	Statements	Branches	How it tests
Backend	Vitest + Fastify `inject()`	72	99%	97%	Real HTTP requests against a freshly seeded PostgreSQL test database
Frontend	Vitest + React Testing Library + jsdom	53	95.4%	90.42%	Renders each page and clicks through it like a user, with the API mocked
SE262	—	**0**	**0%**	**0%**	—

Coverage thresholds as a local gate. `npm run coverage` fails if coverage drops below the thresholds in `backend/vitest.config.js` (statements 85%, branches 75%) and `frontend/vite.config.js` (statements 70%, branches 60%). CI runs the same command, so a pull request that lowers coverage turns red.

What the tests cover, for example: login by e-mail or username; stock limits in the cart; checkout assigning real keys and rolling back when keys run out; one review per user per game; admin access blocked for customers (403) and guests (401); product image upload; deleting a sold key refused; and every page's loading, empty and error states.

Gap found by coverage: `App.jsx`, which only lists the page routes, is the one frontend file at 0%.

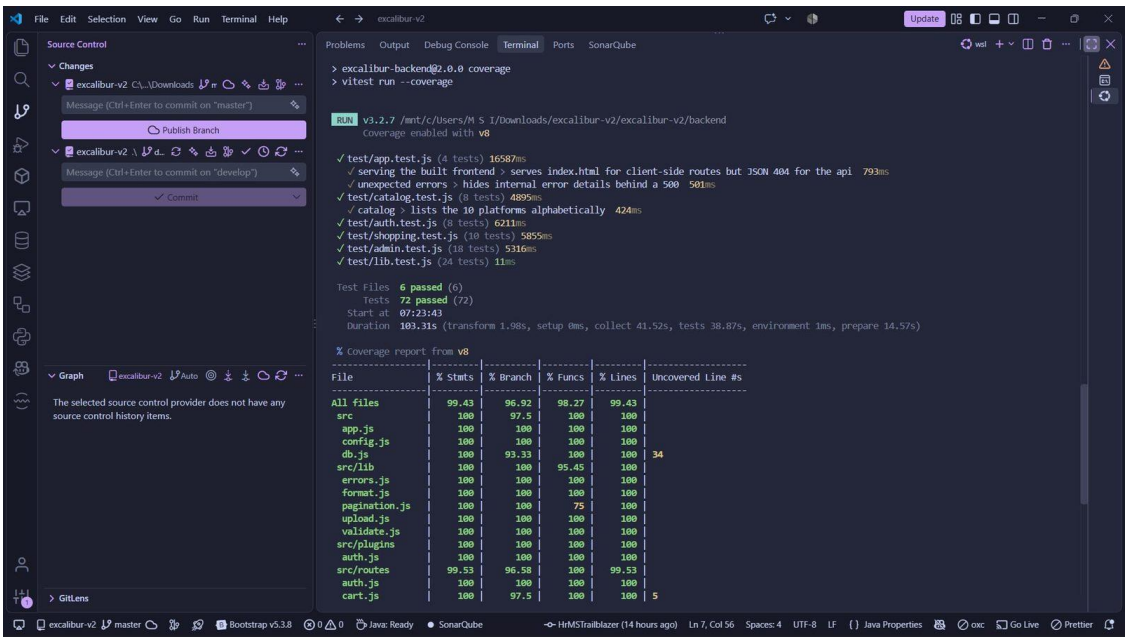


Figure 12.1: backend npm run coverage: 6 test files, 72 tests passed; 99.43% statements, 96.92% branches.

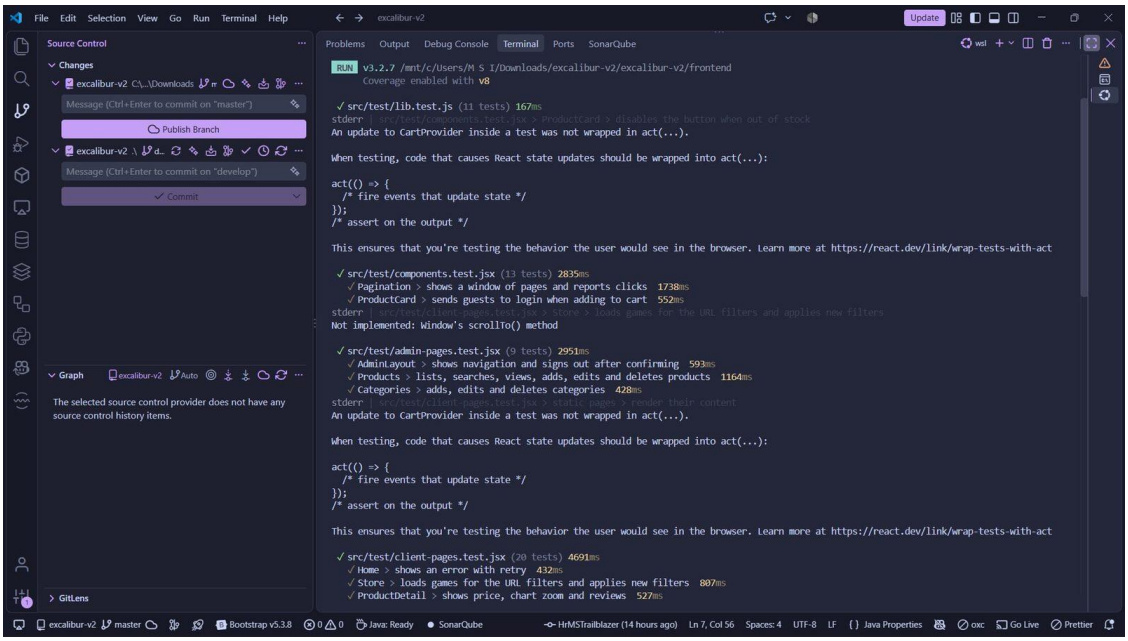


Figure 12.2: frontend tests running with Vitest and React Testing Library.

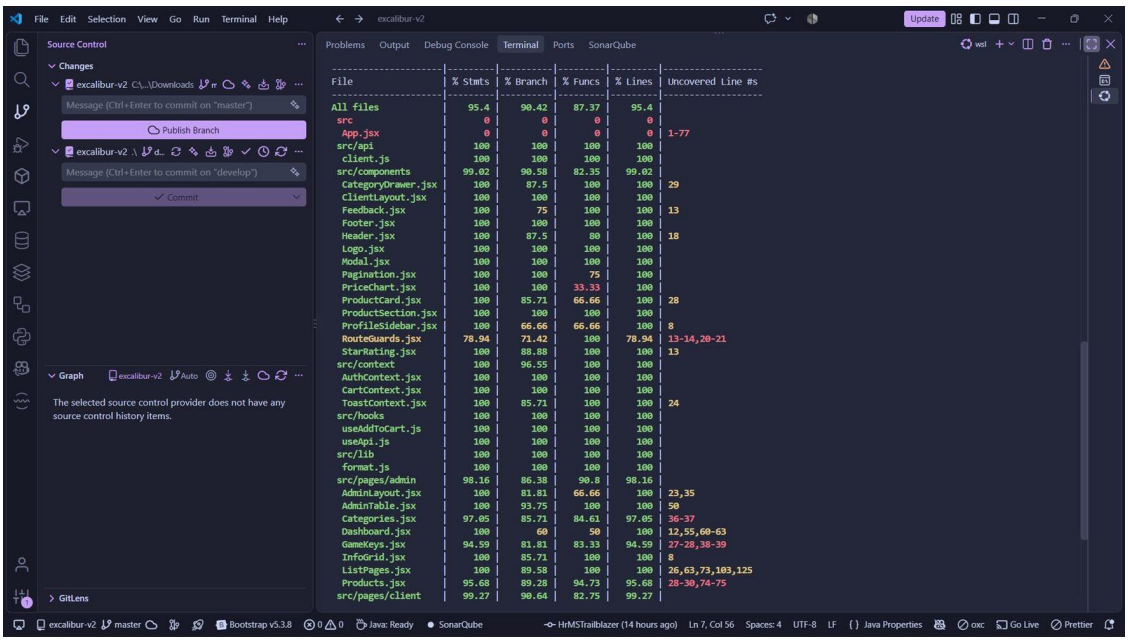


Figure 12.3: frontend coverage table: 95.4% statements, 90.42% branches; `App.jsx` is the only file at 0%.

13. Process: SonarQube and the quality gate

SonarQube found 4 security problems and 1 reliability problem in the first scans of the new code; we fixed or reviewed every one, and the new frontend moved from a failed to a passed quality gate while the SE262 code still fails it.

Our quality gate (“Excalibur Gate”, on overall code)

Condition	Fails when
Coverage	is less than 80%
Duplicated lines	is greater than 3%
Security rating	is worse than A
Reliability rating	is worse than A

We chose overall code rather than SonarQube’s default “new code” gate because on a first scan nothing counts as new, so the default gate passes everything and proves nothing.

Analysis 1: UI and frontend (Piyachat)

Metric	SE262 (old)	SE327 frontend, first scan	SE327 frontend, after fixes
Lines of code	6k	2.3k	2.3k
Security	C (9 issues)	C (2)	**A (0)**
Reliability	C (39 issues)	C (1)	**A (0)**
Maintainability	A (77 issues)	A (17)	A (16)
Coverage	0.0%	94.6%	94.6%
Duplications	2.7%	0.0%	0.0%
Quality gate	**Failed**	**Failed**	**Passed**

The three findings and fixes:

1. **Security:** `Math.random()` used for toast message IDs (`ToastContext.jsx`) → replaced with a counter.
2. **Security:** `Math.random()` used for the payment reference number (`Checkout.jsx`) → replaced with `crypto.getRandomValues()`.
3. **Reliability:** `tabIndex` on a non-interactive `<ul>` in the account menu (`Header.jsx`) → removed, which also fixed one maintainability issue.

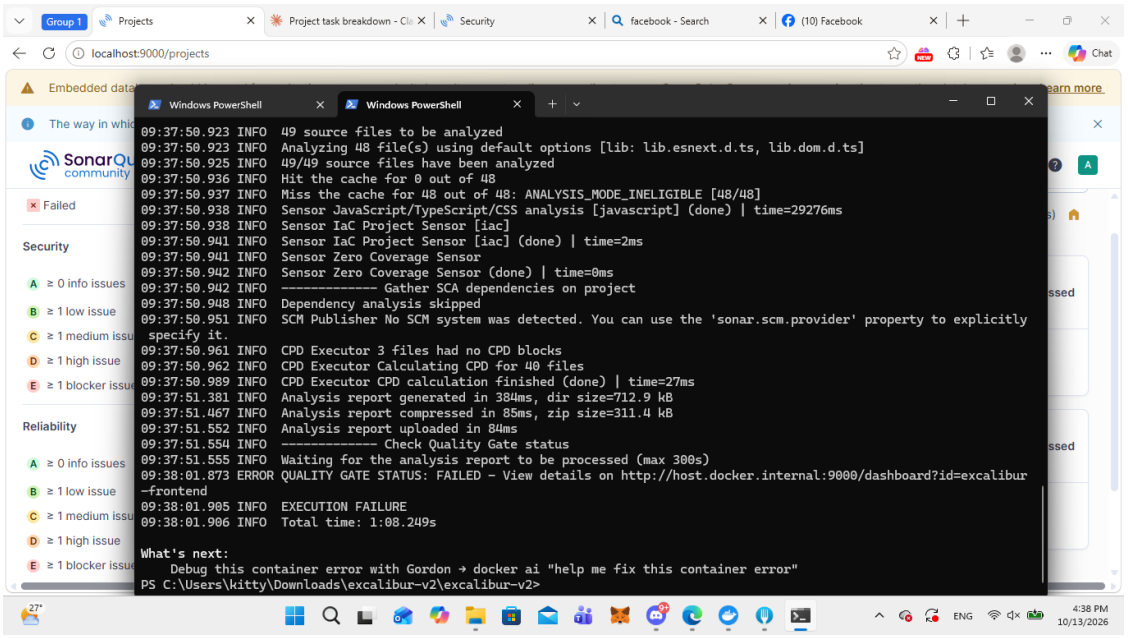


Figure 13.1: first scan of the new frontend with Excalibur Gate: `QUALITY GATE STATUS: FAILED`.

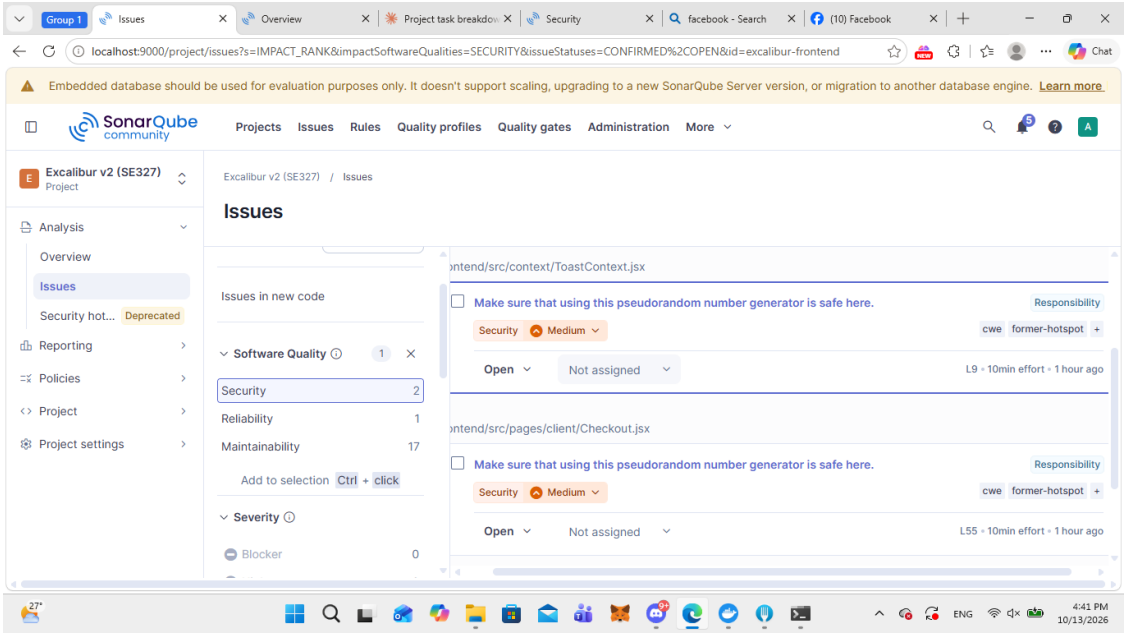


Figure 13.2: the 2 security findings: pseudorandom number generator in `ToastContext.jsx` (line 9) and `Checkout.jsx` (line 55).

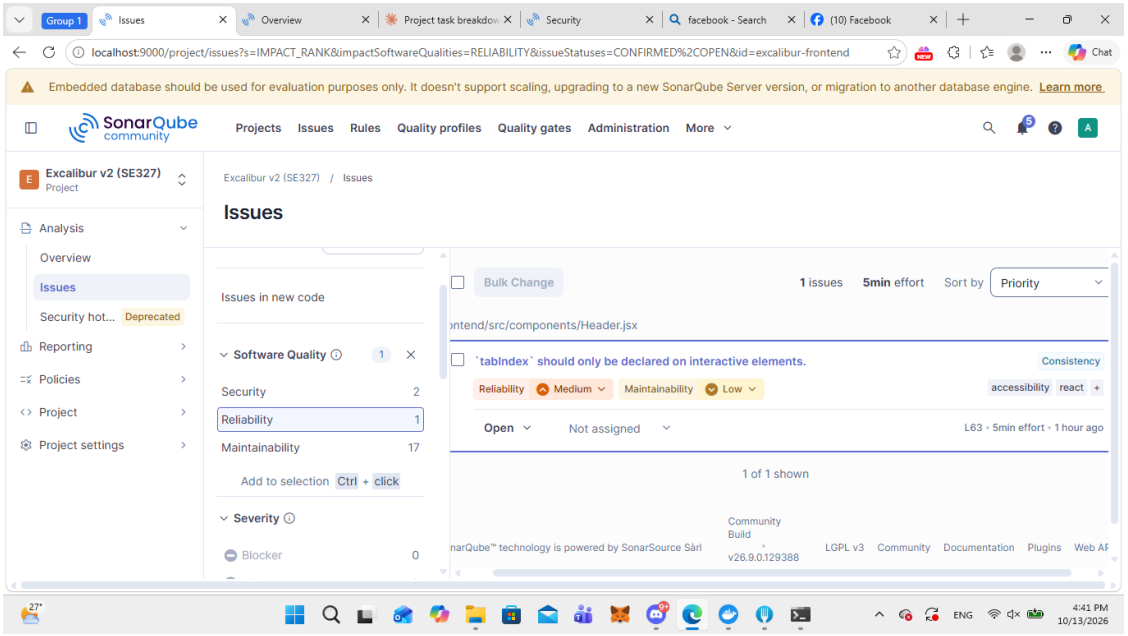


Figure 13.3: the reliability finding: `tabIndex` on a non-interactive element in `Header.jsx` (line 63).

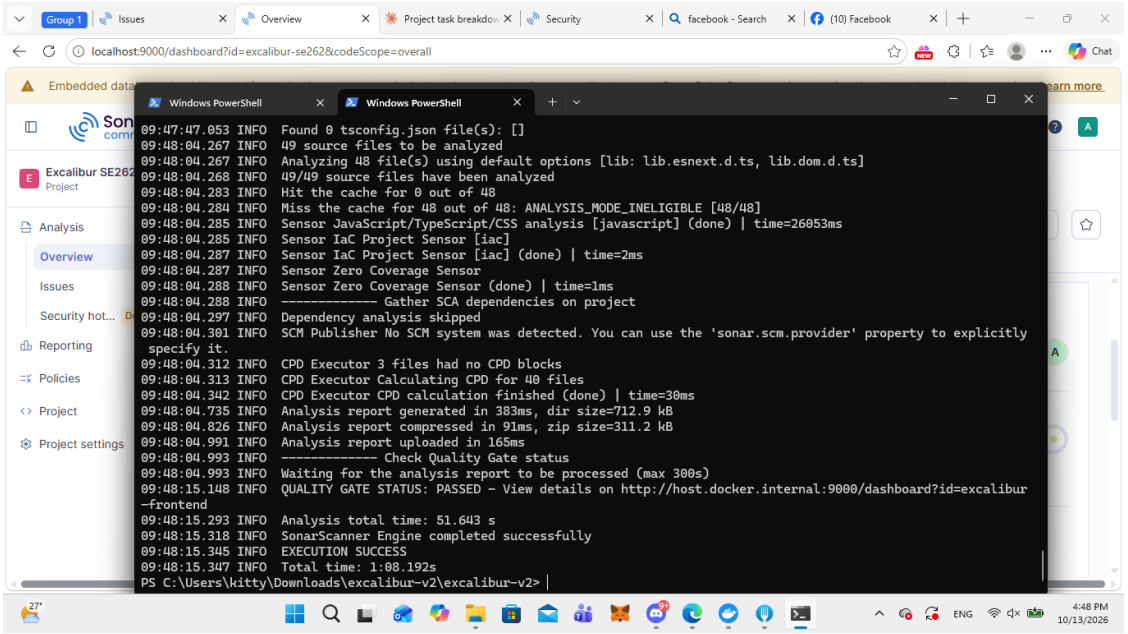


Figure 13.4: scan after the three fixes: `QUALITY GATE STATUS: PASSED`, `EXECUTION SUCCESS`.

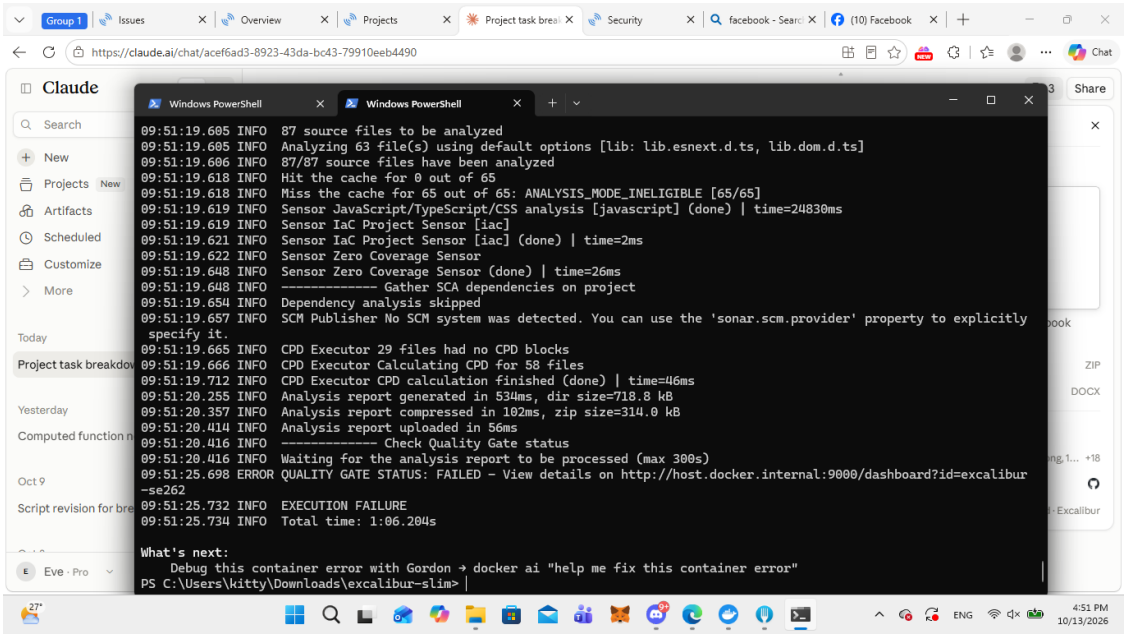


Figure 13.5: the SE262 code scanned with the same gate: `QUALITY GATE STATUS: FAILED`.

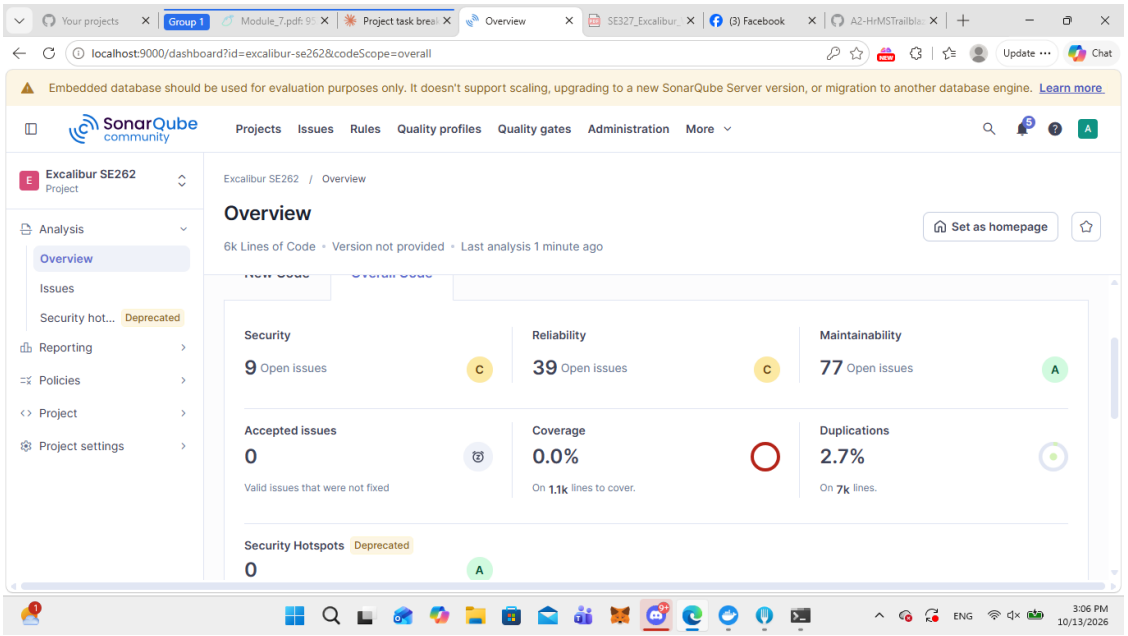


Figure 13.6: SE262 overview: 6k lines, Security C (9), Reliability C (39), Maintainability A (77), 0.0% coverage, 2.7% duplications.

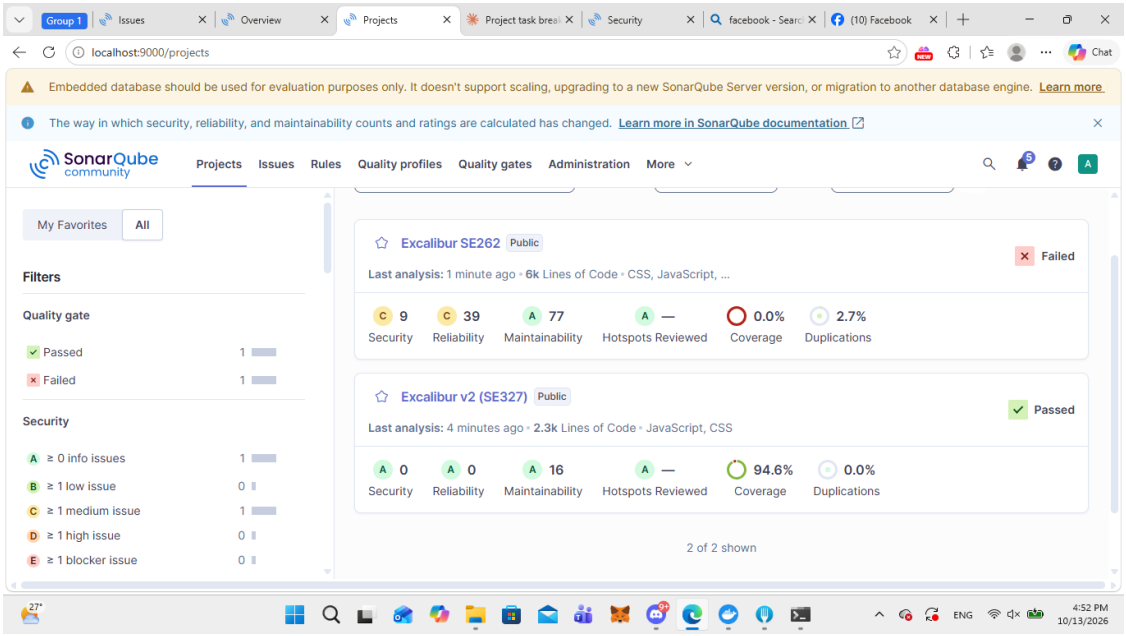


Figure 13.7: projects page side by side: SE262 fails the gate; the new version passes with Security A (0), Reliability A (0) and 94.6% coverage.

Analysis 2: whole project, backend and frontend (Aung Khaing Phyo)

Metric	SE262 (old)	SE327, first scan	SE327, after review
Lines of code	1.5k	3.3k	3.3k
Security	C (1 issue)	**E (4 issues)**	**A (0)**
Reliability	C (21 issues)	C (1)	C (1)
Maintainability	A (32 issues)	A (19)	A (14)
Coverage	0.0% (701 lines)	81.3% (2.6k lines)	95.9% (3.2k lines)
Duplications	0.0%	0.0%	0.0%

- The 4 security findings were reviewed one by one and marked **Accepted** in SonarQube (valid but not fixed). *Two of them were the frontend Math.random findings, which PR #1 later fixed in code, so 2 accepted findings remain (see the table and Figure 13.9b below).*
- Maintainability dropped from 19 to 14 after the refactor commits (optional chaining, `void` on navigation promises).
- Coverage rose to 95.9% once coverage data covered 3.2k lines instead of 2.6k.
- The remaining reliability issue was the `tabIndex` finding fixed in Analysis 1; after PR #1 was merged, reliability became A (Figure 13.9a).

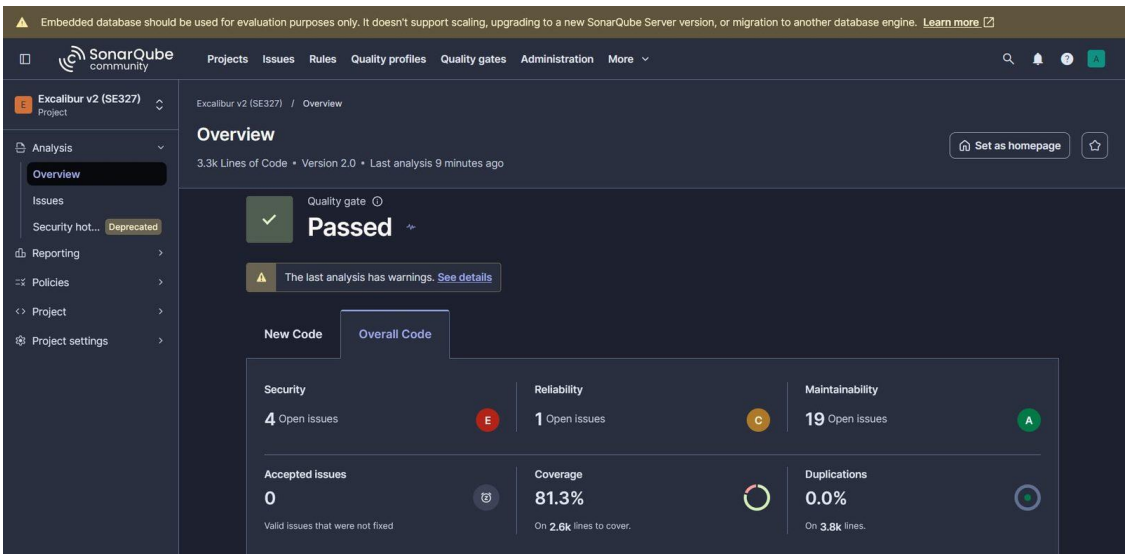


Figure 13.8: whole project, first scan: Security E (4 issues), Reliability C (1), Maintainability A (19), 81.3% coverage, 0.0% duplications.

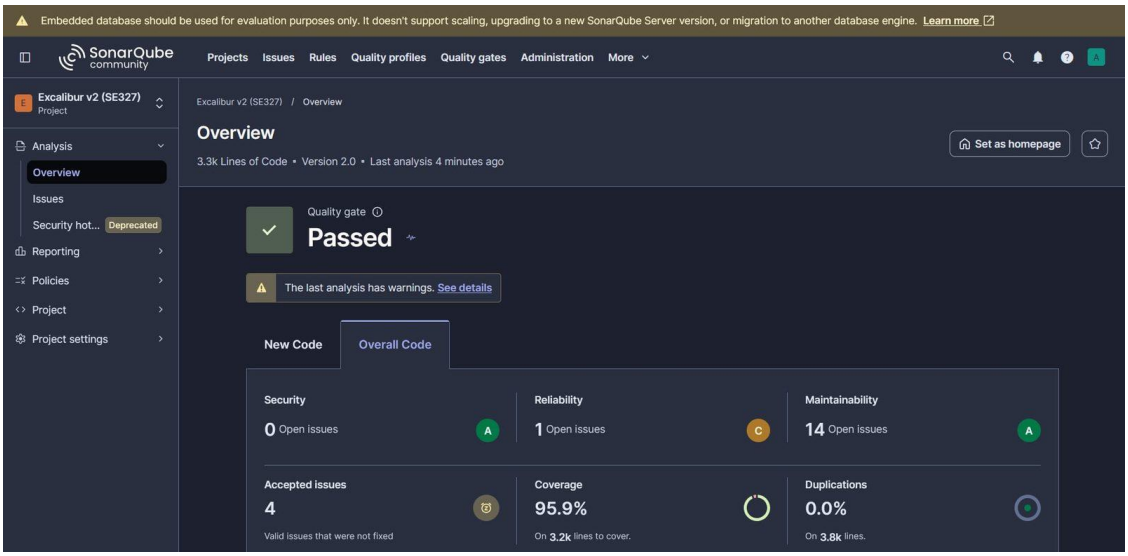


Figure 13.9: whole project after review: Security A (0), Reliability C (1), Maintainability A (14), 4 accepted issues, 95.9% coverage.

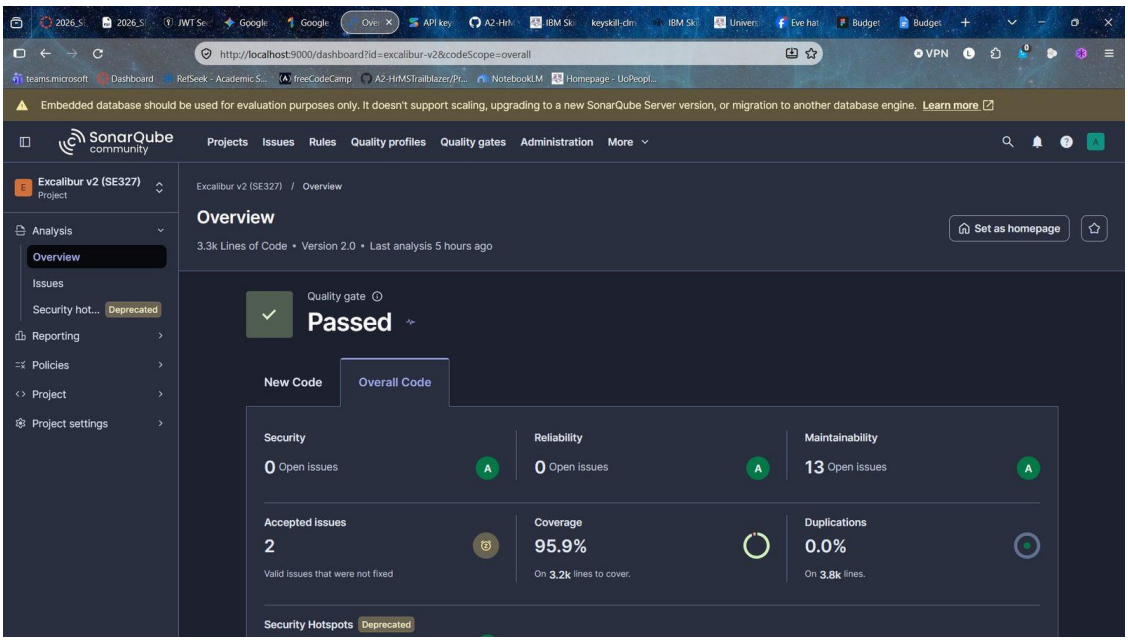


Figure 13.9a: whole project, Overall Code tab, after PR #1 and PR #2 were merged (version 2.0, 3.3k lines). Quality gate Passed; Security A (0), Reliability A (0), Maintainability A (13), 2 accepted issues, coverage 95.9%, duplications 0.0%. The Reliability issue left in Figure 13.9 is gone after Piyachat's frontend fix.

Accepted findings and reasons

File and line	SonarQube finding	Severity	Why it is accepted
`backend/src/config.js`, L20	Make sure this PostgreSQL password gets changed and removed from the code.	Security, Blocker	The PostgreSQL password has already been changed. The value in the code is only a local development fallback, and the real connection string is supplied through the `DATABASE_URL` environment variable.
`backend/src/lib/upload.js`, L30	Make sure that using this pseudorandom number generator is safe here.	Security, Medium	`Math.random()` is only used for simple generation of a temporary upload file name, not for passwords, tokens or anything security-sensitive, so a pseudorandom number is safe here.

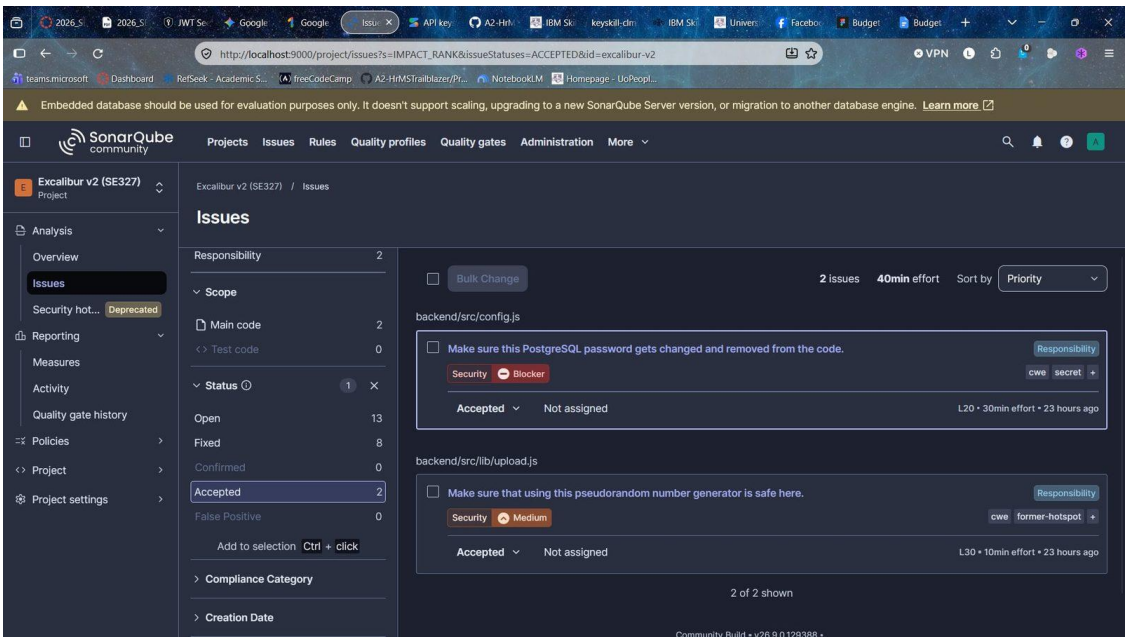


Figure 13.9b: the 2 accepted issues in SonarQube (Status = Accepted). The other filters show 13 open maintainability issues and 8 fixed.

The two analyses count old lines of code differently (6k vs 1.5k) because they used different file exclusions; each compares old and new with the same settings.

How the gate improved our process: without the gate, all scans showed “Passed” and the security findings would have shipped. With it, a failed scan stopped the frontend until the exact lines SonarQube named were fixed, and the result was verified by scanning again.

Final whole-project scan after merging (14 October 2026)

The last scan shows 0 new issues, 95.9% coverage on 3.2k new lines (the gate requires at least 80%) and 0.0% duplications (the gate requires at most 3%). Version 2.0 was released on 14 October 2026 with the quality gate passing, so SonarQube’s quality gate history marks it as a safe release (1 of 1).

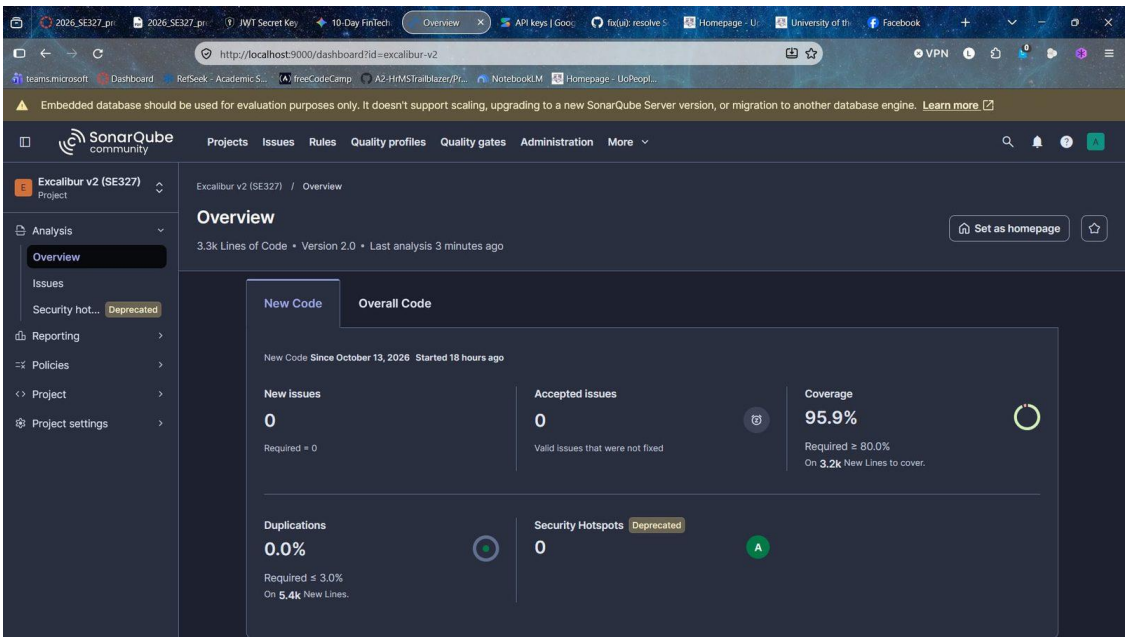


Figure 13.10: Excalibur v2 (SE327), new code since 13 October 2026: 0 new issues, 95.9% coverage, 0.0% duplications, all within the gate's limits.

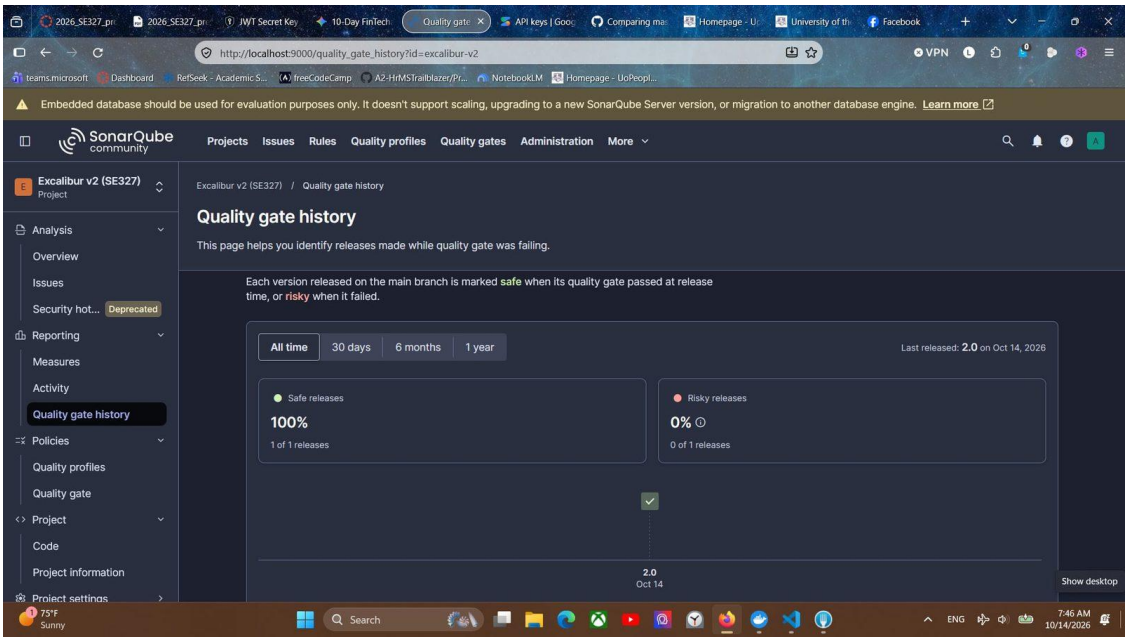


Figure 13.11: quality gate history: release 2.0 (14 October 2026) passed the gate, 100% safe releases.

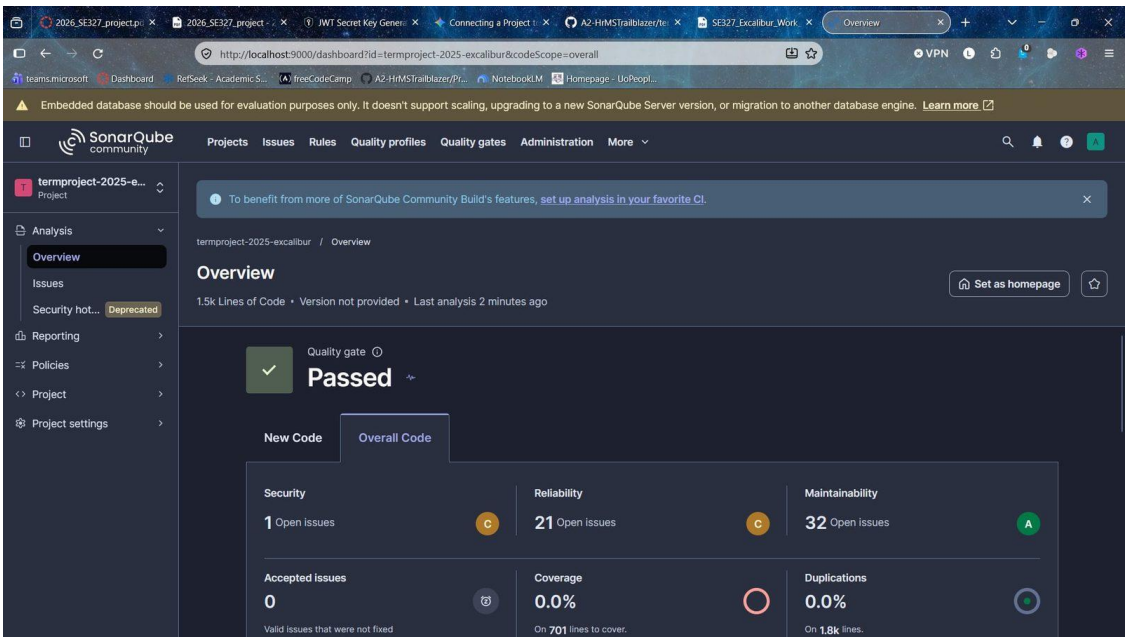


Figure 13.12: the original SE262 project (termproject-2025-excalibur): 1.5k lines, Security C (1 issue), Reliability C (21), Maintainability A (32), 0.0% coverage on 701 lines.

14. Reflection: the four components before and after

The new stack is smaller, safer and measurable, at the cost of more upfront design (an API) and a stricter database; the survey predicted most strengths correctly but under-estimated the work of splitting frontend from backend.

Component	SE262: strengths → weaknesses	SE327: strengths → weaknesses	Did the survey predict it?
UI	Bootstrap: familiar, quick to start → 3,108 lines of custom CSS in 21 files, colours repeated everywhere	Tailwind + DaisyUI: 57 lines of theme tokens, two themes, nearly identical look → long class lists are harder to read	Yes. Speed and theming were as expected; readability was worse than we thought
Frontend	EJS: simple, no API needed → page reloads, scripts that parsed whole HTML pages, and templates SonarQube cannot analyse	React: instant updates, testable pages (53 tests, 95.4% / 90.42%), fully analysed → needed an API, loading states and careful hook rules	Partly. Testing and tooling were better than expected; the API split took more time than predicted
Backend	Express: minimal and well known → no validation, plain-text passwords, client-supplied prices, three admin actions that always failed	Fastify: schema validation, encrypted sessions, `inject()` tests (72, 99% / 97%) → plugin rules and error formatting took time to learn	Yes. Porting was fast because routes look like Express
Database	MySQL: forgiving → the forgiving behaviour hid	PostgreSQL: strict types, views, `SKIP LOCKED` for safe	Mostly. The strictness found real

Component	SE262: strengths → weaknesses	SE327: strengths → weaknesses	Did the survey predict it?
	bugs (case-insensitive enums, a dropped column still used) and seven fix-up `ALTER`s	checkout, a clean schema → reserved words, `$1` placeholders and `NUMERIC` as strings	bugs, but the `NUMERIC` surprise was not in our survey

Overall numbers

Measure	SE262	SE327
Application code	8,429 lines	about 3,350 lines
Automated tests	0	125
Quality gate (Excalibur Gate)	Failed	Passed
Known security problems	Plain-text passwords, client-chosen prices, hard-coded session secret	None open; 2 reviewed and accepted

15. Reflection: recommended stack for a similar project

For a small team building a web store like Excalibur, we would choose the same four technologies again, with two changes in how we use them.

Component	Recommendation	Why, based on this project
UI framework	**Tailwind CSS + DaisyUI**	Cut custom CSS from 3,108 to 57 lines and made theming trivial. Keep long class lists readable by extracting small components (as we did with `ProductCard` and `Modal`).
Frontend	**React with Vite**, adding **TypeScript**	React gave the best testing and SonarQube results. TypeScript would have caught the `NUMERIC`-as-string and response-shape mistakes before running the code.
Backend	**Fastify**	Fast to port from Express, built-in validation, and `inject()` made 72 integration tests run in seconds. Share the JSON schemas with the frontend so both sides agree on the API.
Database	**PostgreSQL**	Relational data plus safe concurrent checkout. Use a migration tool from day one instead of a single schema file, so changes can be reviewed in pull requests.